Računarska grafika
(3OER7O02)

Crtanje primitiva

Vežbe

Crtanje tačke

Crtanje tačke

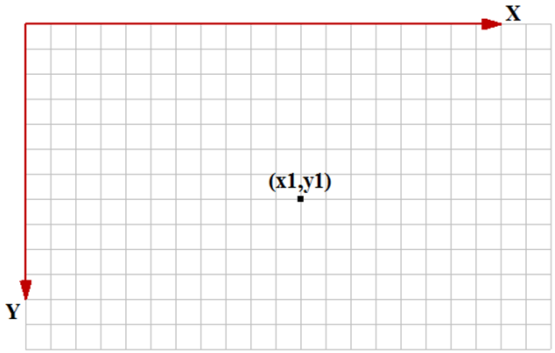
- Vrednost piksela u tački sa zadatim koordinatama se pribavlja/postavlja na zadatu boju

  COLORREF CDC::GetPixel(int x, int y) const;

  COLORREF CDC::GetPixel(POINT point) const;

  COLORREF CDC::SetPixel(int x, int y, COLORREF crColor);

  COLORREF CDC::SetPixel(POINT point, COLORREF crColor);



- Parametri metoda
  - ***x*** – logička x-koordinata tačke
  - ***y*** – logička y-koordinata tačke
  - ***point*** – (x,y) koordinate su definisane strukturom POINT ili klasom CPoint
  - ***crColor*** – boja na koju se postavlja piksel

```
int x1 = 110, y1 = 70;
COLORREF prevCol = pDC->SetPixel(x1, y1, RGB(0, 0, 0));
```

Crtanje linija i mnogouglova

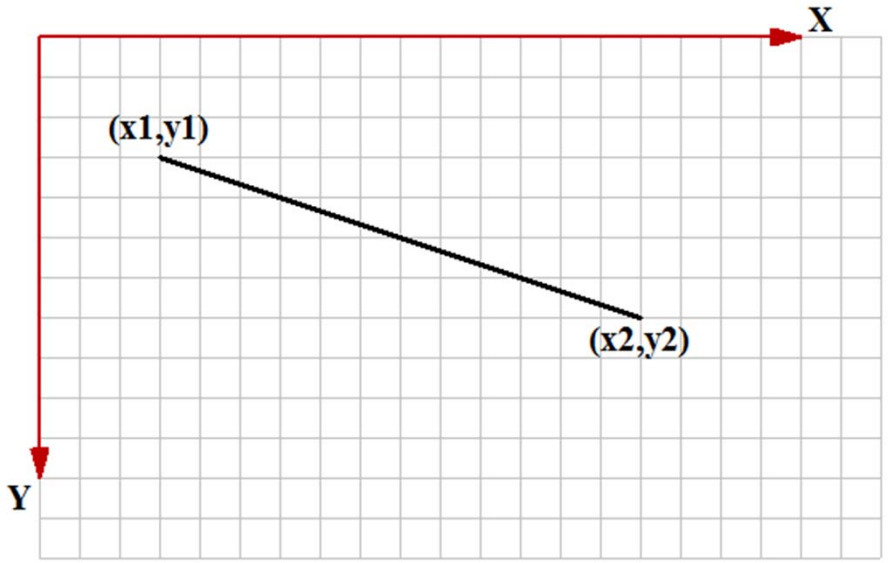
Crtanje linije

CPoint CDC::MoveTo(int x, int y);

CPoint CDC::MoveTo(POINT point);

CPoint CDC::LineTo(int x, int y);

CPoint CDC::LineTo(POINT point);




```
int x1 = 75, y1 = 75, x2 = 375, y2 = 175;
pDC->MoveTo(x1, y1);
pDC->LineTo(x2, y2);
```

Crtanje poligonalne linije

- Crtanje jedne izlomljene linije

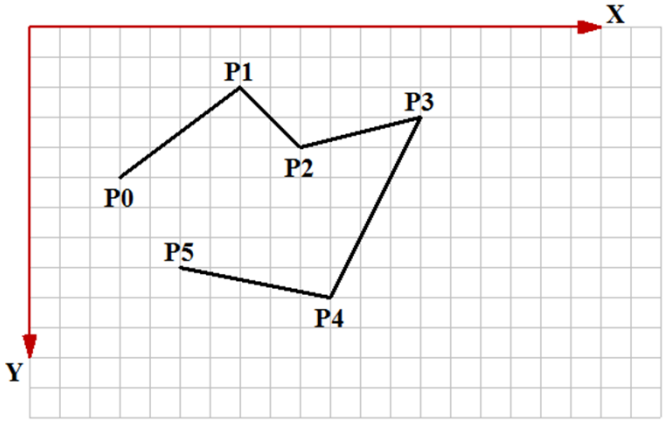
  BOOL CDC::Polyline(LPPOINT *lpPoints*, int *nCount*);

  - ***lpPoints*** – niz temena izlomljene linije
  - ***nCount*** – broj temena izlomljene linije

- Crtanje niza izlomljenih linija

  BOOL CDC::PolyPolyline(const POINT* *lpPoints*, const DWORD* *lpPolyPoints*, int *nCount*);

  - ***lpPoints*** – niz temena svih izlomljenih linija
  - ***lpPolyPoints*** – niz brojeva, gde je svaki element broj tačaka jedne izlomljene linije
  - ***nCount*** – broj izlomljenih linija (mora biti najmanje 2)




```
POINT pts[6] = {CPoint(100,150), CPoint(200,75),  CPoint(250,125), CPoint(350,100), CPoint(275,250), CPoint(150,225) };
pDC->Polyline(pts, 6);
```

Crtanje mnogougla

- Crtanje jednog poligona

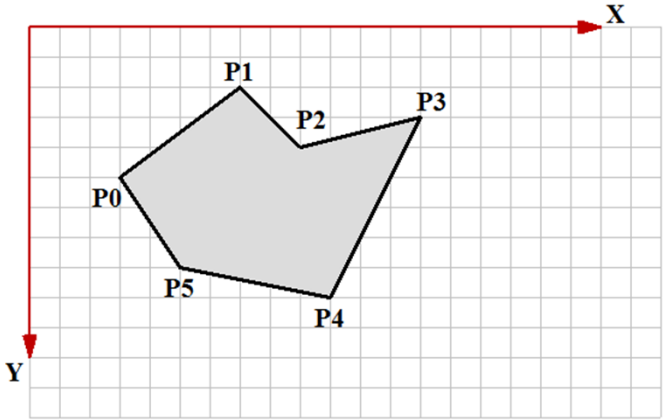
  BOOL CDC::Polygon(LPPOINT *lpPoints*, int *nCount*);

  - ***lpPoints*** – niz temena mnogougla
  - ***nCount*** – broj temena mnogougla

- Crtanje niza poligona

  BOOL CDC::PolyPolygon(const POINT* *lpPoints*, const DWORD* *lpPolyPoints*, int *nCount*);

  - ***lpPoints*** – niz temena svih mnogouglova
  - ***lpPolyPoints*** – niz brojeva, gde je svaki element broj tačaka jednog mnogougla
  - ***nCount*** – broj mnogouglova (mora biti najmanje 2)




```
POINT pts[6] = {CPoint(100,150), CPoint(200,75),  CPoint(250,125), CPoint(350,100), CPoint(275,250), CPoint(150,225) };
pDC->Polygon(pts, 6);
```

Crtanje pravougaonika i elipsi

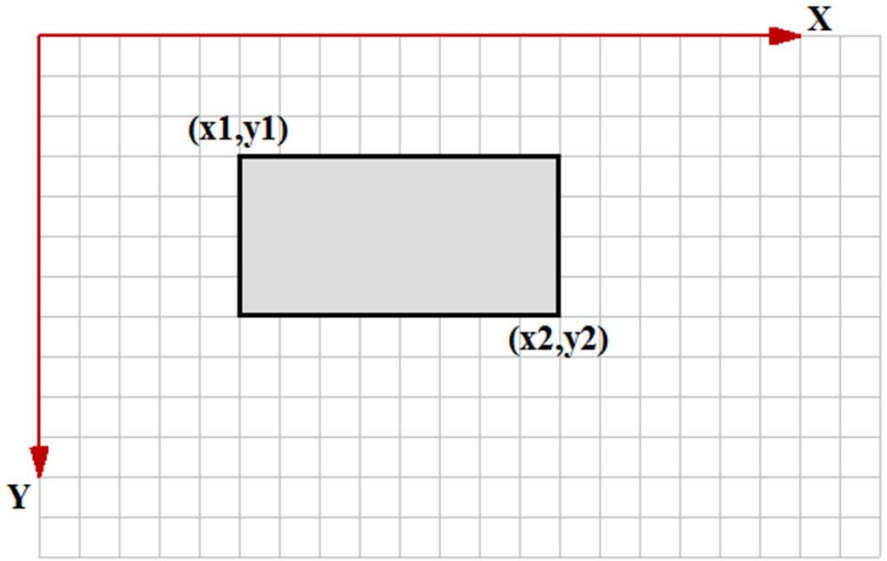
Crtanje pravougaonika

- Crtanje pravougaonika paralelnog koordinatnim osama

BOOL CDC::Rectangle(int *x1*, int *y1*,
int *x2*, int *y2*);

BOOL CDC::Rectangle(LPCRECT *lpRect*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200;
pDC->Rectangle(x1,y1,x2,y2);
```

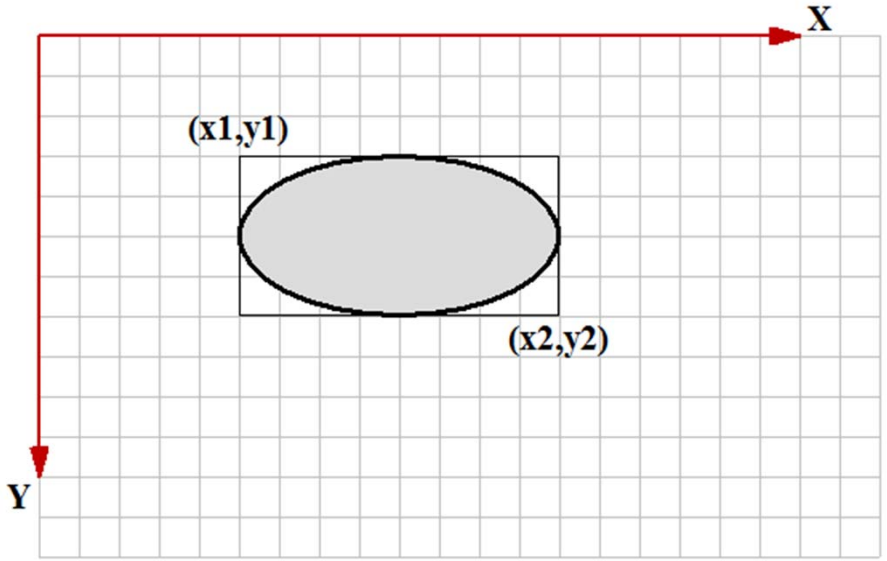
Crtanje elipse

- Crtanje elipse paralelne koordinatnim osama

BOOL CDC::Ellipse(int *x1*, int *y1*,
int *x2*, int *y2*);

BOOL CDC::Ellipse(LPCRECT *lpRect*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200;
pDC->Ellipse(x1, y1, x2, y2);
```

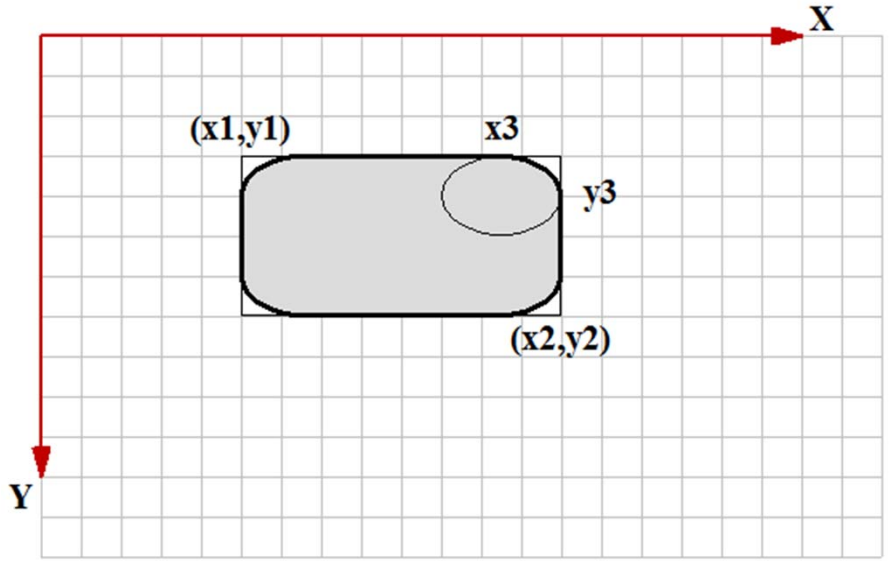
Crtanje zaobljenog pravougaonika

- Crtanje zaobljenog pravougaonika paralelnog koordinatnim osama

BOOL CDC::RoundRect(int *x1*, int *y1*, int *x2*, int *y2*,
int *x3*, int *y3*);

BOOL CDC::RoundRect(LPCRECT *lpRect,*
POINT *point*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**
- Parametar ***point*** je tipa **POINT** na čije mesto se može proslediti argument tipa **CPoint**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200, x3 = 75, y3 = 50;
pDC->RoundRect(x1, y1, x2, y2, x3, y3);
```

Crtanje lukova i lučnih oblika

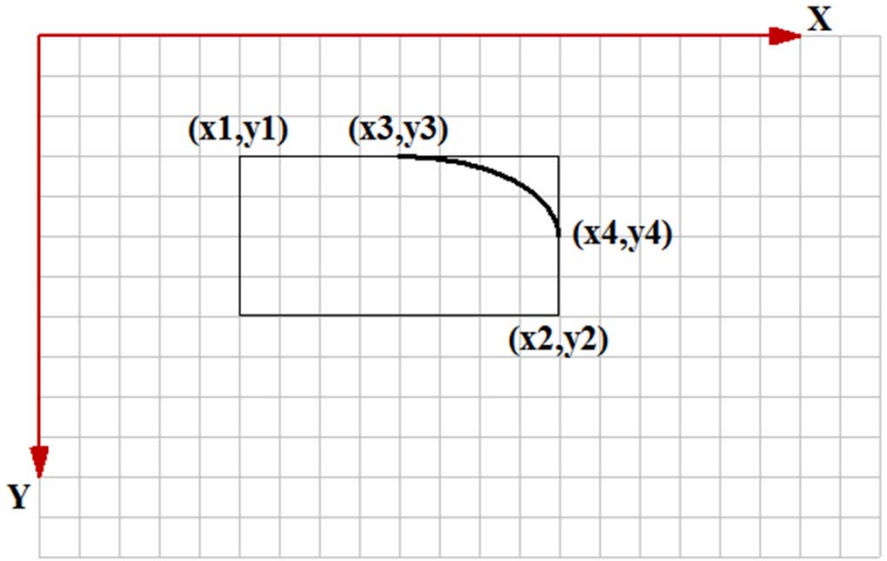
Crtanje luka

- Crtanje luka elipse paralelne koordinatnim osama

BOOL CDC::Arc(int *x1*, int *y1*, int *x2*, int *y2*,
int *x3*, int *y3*, int *x4*, int *y4*);

BOOL CDC::Arc(LPCRECT *lpRect,*
POINT *ptStart*, POINT *ptEnd*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**
- Parametri ***ptStart i ptEnd*** su tipa **POINT** na čije mesto se mogu proslediti argumenti tipa **CPoint**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200,
    x3 = 250, y3 = 100, x4 = 350, y4 = 150;
pDC->Arc(x1, y1, x2, y2, x3, y3, x4, y4);
```

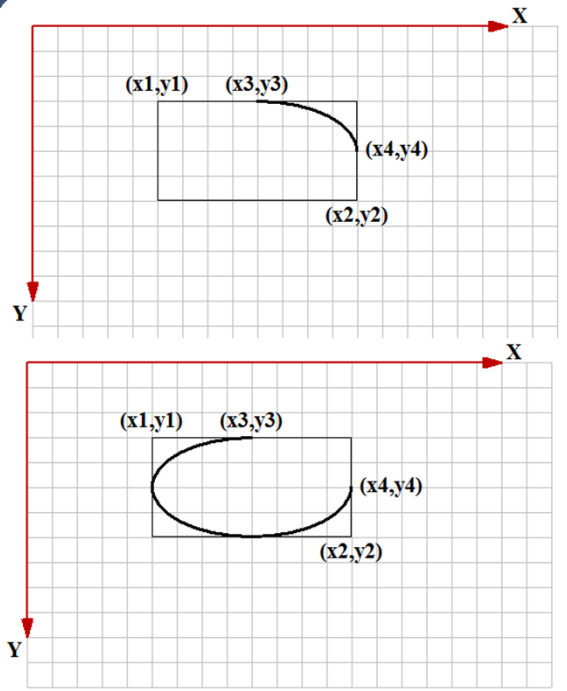
Orijentacija luka

- Orijentacija luka se pribavlja/postavlja na zadatu vrednost

  int CDC::GetArcDirection( );

  int CDC::SetArcDirection(int *nArcDirection*);

- ***nArcDirection*** – orijentacija luka može imati dve vrednosti:
  - **AD_COUNTERCLOCKWISE** – orijentacija suprotna od smera kretanja kazaljke na časovniku
  - **AD_CLOCKWISE** – orijentacija u smeru kretanja kazaljke na časovniku
- Podrazumevana orijentacija je suprotna od smera kretanja kazaljke na časovniku



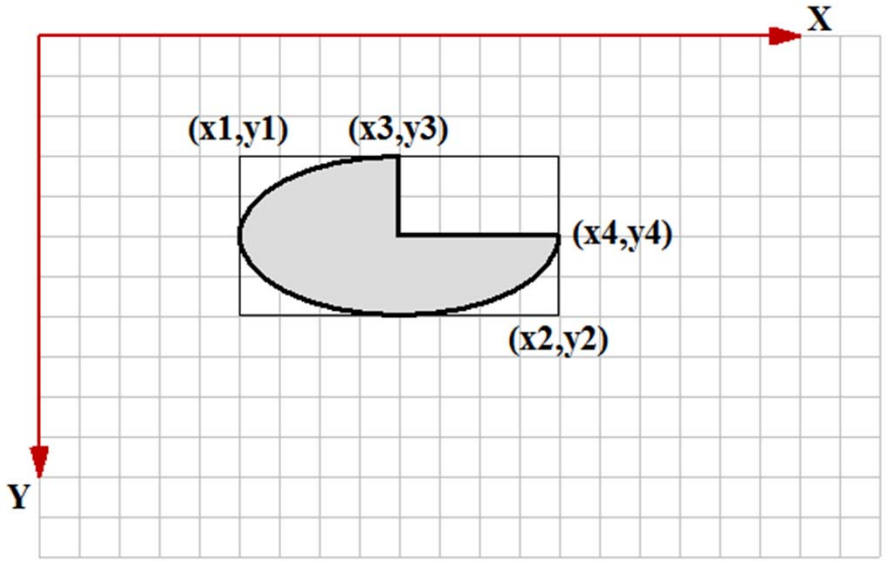
Crtanje pite

- Crtanje pite kao dela elipse paralelne koordinatnim osama

BOOL CDC::Pie(int *x1*, int *y1*, int *x2*, int *y2*,
int *x3*, int *y3*, int *x4*, int *y4*);

BOOL CDC::Pie(LPCRECT *lpRect,*
POINT *ptStart*, POINT *ptEnd*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**
- Parametri ***ptStart i ptEnd*** su tipa **POINT** na čije mesto se mogu proslediti argumenti tipa **CPoint**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200,
    x3 = 250, y3 = 100, x4 = 350, y4 = 150;
pDC->Pie(x1, y1, x2, y2, x3, y3, x4, y4);
```

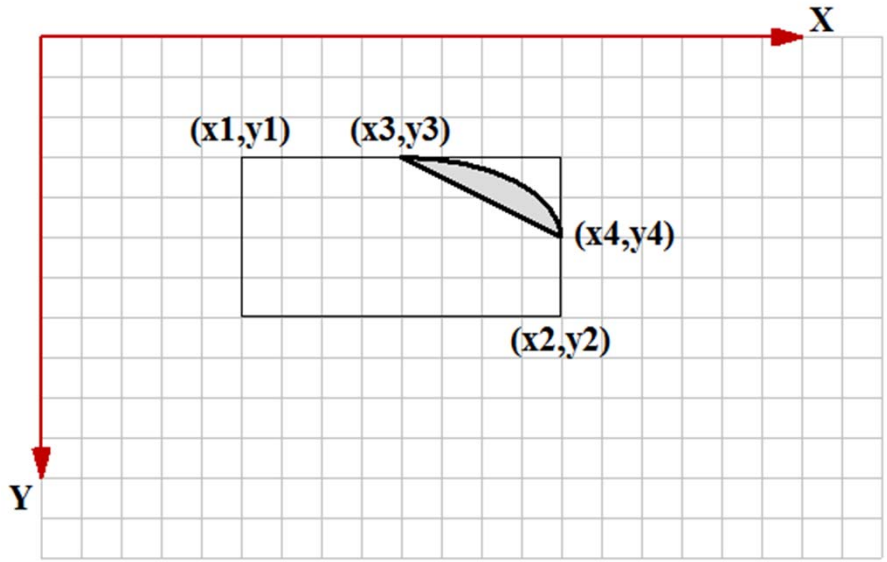
Crtanje odsečka

- Crtanje odsečka elipse paralelne koordinatnim osama

BOOL CDC::Chord(int *x1*, int *y1*, int *x2*, int *y2*,
int *x3*, int *y3*, int *x4*, int *y4*);

BOOL CDC:: Chord(LPCRECT *lpRect,*
POINT *ptStart*, POINT *ptEnd*);

- Parametar ***lpRect*** je tipa **LPCRECT** na čije mesto se može proslediti argument tipa **CRect**
- Parametri ***ptStart i ptEnd*** su tipa **POINT** na čije mesto se mogu proslediti argumenti tipa **CPoint**




```
int x1 = 150, y1 = 100, x2 = 350, y2 = 200,
    x3 = 250, y3 = 100, x4 = 350, y4 = 150;
pDC->SetArcDirection(AD_CLOCKWISE);
pDC->Chord(x1, y1, x2, y2, x3, y3, x4, y4);
```

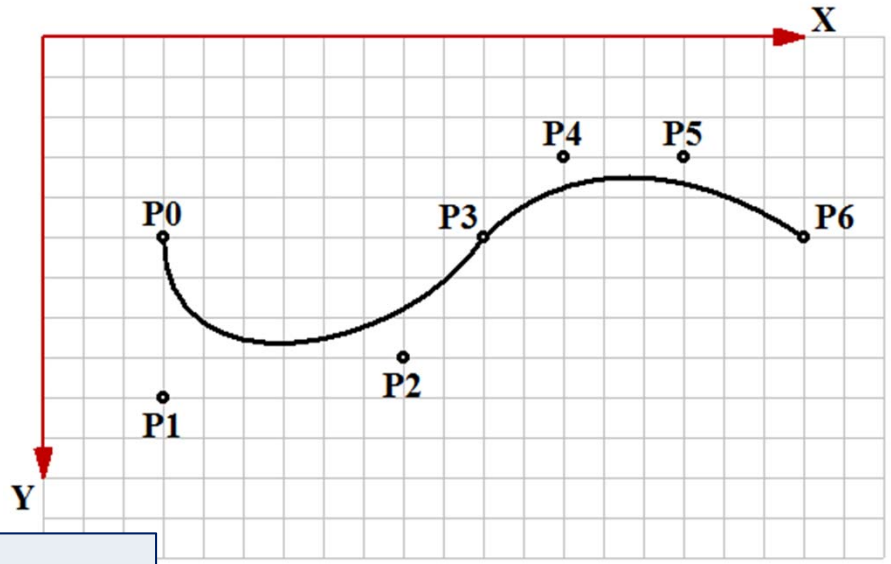
Crtanje Bezier-ove krive

Crtanje Bezier-ove krive

- Crtanje krive linije definisane *Bezier*-ovom krivom

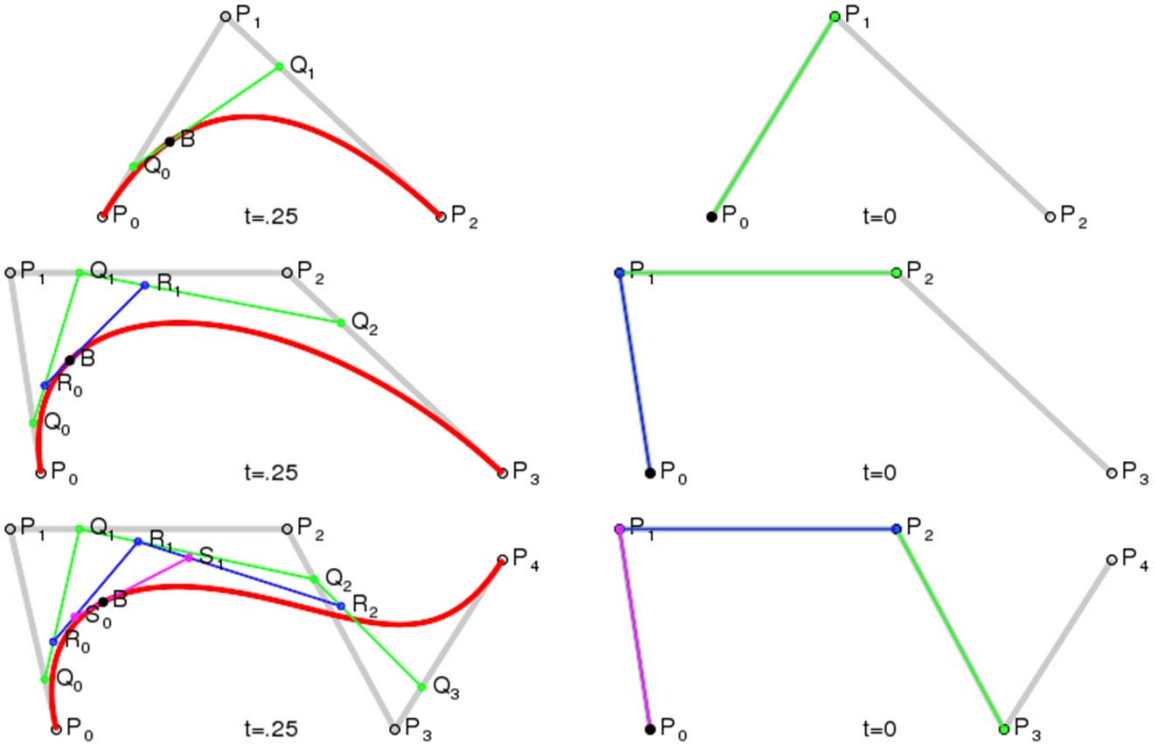
  BOOL CDC::PolyBezier(const POINT* lpPoints, int *nCount*);

- ***lpPoints*** – niz tačaka (kombinovano, krajnje i kontrolne)
- ***nCount*** – broj tačaka u nizu
- Parametar ***lpPoints*** je tipa **POINT*** na čije mesto se može proslediti argument tipa **CPoint***




```
POINT pts4[7] = {CPoint(100,150), CPoint(100,250),
                 CPoint(250,225), CPoint(300,150),
                 CPoint(350,100), CPoint(425,100), CPoint(500,150)};
pDC->PolyBezier(pts4, 7);
```

Bezier-ova kriva



Bezier-ova kriva

- Bezier-ova kriva može se predstaviti u parametarskom obliku:

$$x(t) = a_x t^3 + b_x t^2 + c_x t + x_0$$

$$y(t) = a_y t^3 + b_y t^2 + c_y t + y_0$$

- Ukoliko su poznati koeficijenti a_x, b_x , c_x i početna tačka $T_0(x_0,y_0)$, tačke krive $T_1(x_1,y_1)$, $T_2(x_2,y_2)$ i $T_3(x_3,y_3)$ se dobijaju rešavanjem sledećih jednačina:

$$x_1 = x_0 + c_x/3$$

$$x_2 = x_1 + (c_x + b_x)/3$$

$$x_3 = x_0 + a_x + b_x + c_x$$

Bezier-ova kriva

- Bezier-ova kriva može se predstaviti u parametarskom obliku:

$$x(t) = a_x t^3 + b_x t^2 + c_x t + x_0$$

$$y(t) = a_y t^3 + b_y t^2 + c_y t + y_0$$

- Ukoliko su poznate tačke $T_0(x_0,y_0)$, $T_1(x_1,y_1)$, $T_2(x_2,y_2)$ i $T_3(x_3,y_3)$, koeficijenti a_x, b_x i c_x se dobijaju rešavanjem sledećih jednačina:

$$c_x = 3(x_1 - x_0)$$

$$b_x = 3(x_2 - 2x_1 + x_0)$$

$$a_x = x_3 + 3(x_1 - x_2) - x_0$$

GDI+

Definisanje krajeva linija

```
Pen pen(Color(255, 0, 0, 255), 8);
stat = pen.SetStartCap(LineCapArrowAnchor);
stat = pen.SetEndCap(LineCapRoundAnchor);
stat = graphics.DrawLine(&pen, 20, 175, 300, 175);
```

Različite primitive za crtanje linija

DrawArc(Pen* pen, Rect& rect, REAL startAngle, REAL sweepAngle)
DrawBezier(Pen* pen, Point& pt1, Point& pt2, Point& pt3, Point& pt4)
DrawBeziers(Pen* pen, Point* points, INT count)
DrawClosedCurve(Pen* pen, Point* points, INT count, REAL tension)
DrawCurve(Pen* pen, Point* points, INT count, INT offset, INT numberOfSegments, REAL tension)
DrawCurve(Pen* pen, Point* points, INT count, REAL tension)
DrawEllipse(Pen* pen, Rect& rect)
DrawEllipse(Pen* pen, INT x, INT y, INT width, INT height)
DrawPath(pen, path)
DrawPie(Pen* pen, Rect& rect, REAL startAngle, REAL sweepAngle)
DrawPolygon(Pen* pen, Point* points, INT count)
DrawRectangle(Pen* pen, Rect& rect)

Četka

- Puna četka

  SolidBrush(const Color& color);

- Četka sa šrafurom

  HatchBrush(HatchStyle hatchStyle, const Color& foreColor, const Color& backColor);

  - Postoji preko 50 predefinisanih tipova šrafura

- Četka sa bitmapom

  TextureBrush(Image* image, WrapMode wrapMode);

  - enum WrapMode { WrapModeTile, WrapModeTileFlipX, WrapModeTileFlipY, WrapModeTileFlipXY, WrapModeClamp };

Četka
Primer

```
Color black(255,0,0,0);
Color white(255,255,255,255);
HatchBrush  brush(HatchStyleHorizontal, black, white);
graphics.FillRectangle(&brush, 20, 20, 100, 50);
HatchBrush brush1(HatchStyleVertical, black, white);
graphics.FillRectangle(&brush1, 120, 20, 100, 50);
HatchBrush brush2(HatchStyleForwardDiagonal, black, white);
graphics.FillRectangle(&brush2, 220, 20, 100, 50);
HatchBrush brush3(HatchStyleBackwardDiagonal, black, white);
graphics.FillRectangle(&brush3, 320, 20, 100, 50);
```

Različite primitive za ispunu figura

FillClosedCurve(Brush* brush, Point* points, INT count)

FillEllipse(Brush* brush, Rect& rect)

FillPath(brush, path)

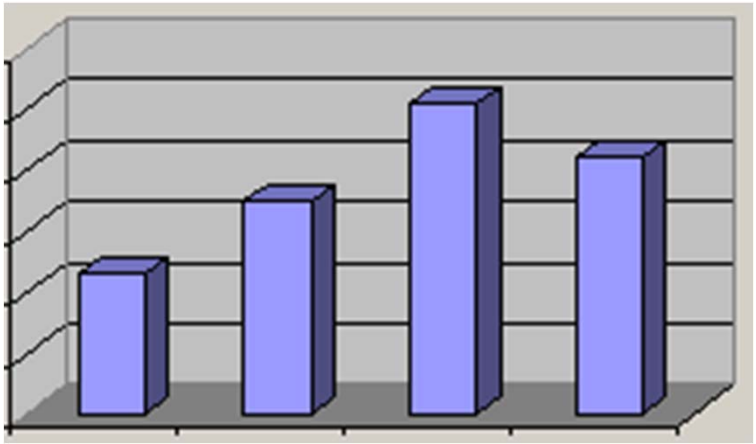
FillPie(Brush* brush, Rect& rect, REAL startAngle, REAL sweepAngle)

FillPolygon(Brush* brush, Point* points, INT count)

FillRegion(brush, region)

Zadatak za vežbu

Napisati funkciju koja iscrtava 3D dijagram sa stupcima (kao u Excel-u). Ulazni parametri f-je su: okvirni pravougaonik koji definiše gde u prozoru treba iscrtati grafikon (u logičkim koordinatama), vektor realnih brojeva kojim se zadaju vrednosti koje treba predstaviti na grafikonu, dužina prethodnog vektora i boja stubaca (njihove prednje strane). Smatrati da su stupci razmaknuti za 1.5 debljina jednog stupca, i da se njihove visine i debljine menjaju zavisno od zadatog okvirnog pravougaonika.



Pitanja